

Azure DevOps + YAML

Complete Guide

History, Architecture, Organization Setup, Agents, Pipelines,
Terraform aur YAML — sab ek hi jagah, revision ke liye.

ORG EXAMPLE

Vistara Technologies

PROJECT EXAMPLE

Vistara Sense

ACCESS

dev.azure.com

01 • History, One-Roof & Tool Mapping	pg 2
02 • Access & Hierarchy	pg 3
03 • Organization Setup – Step by Step	pg 4
04 • Agents & Pipeline Structure	pg 5
05 • Terraform Pipeline & State File	pg 6
06 • YAML Fundamentals – 4 Rules	pg 7
07 • YAML Pipeline & Practice Workflow	pg 8
08 • Homework, Roadmap & Recall	pg 9

• 01 HISTORY – TFS SE AZURE DEVOPS TAK

2019 se pehle: Microsoft ka product **TFS** (Team Foundation Server) kehlata tha, baad mein **VSTS** (Visual Studio Team Services) naam mila.

2019 mein rebrand: Cloud ke saath integrate karke naya naam diya — **Azure DevOps**. Ye ek **Software-as-a-Service (SaaS)** product hai jisme development lifecycle ke saare tools ek chhat ke neeche mil jaate hain.

• 02 "ONE ROOF" CONCEPT – KYUN POPULAR HAI

Normally alag-alag kaam ke liye **alag tool, alag company, alag license** chahiye hota — JIRA kisi aur ka, Jenkins kisi aur ka, Nexus kisi aur ka. Integration aur maintenance ka jhanjhat alag.

Azure DevOps ne in sabko **ek hi product** mein club kar diya — ek baar login karo, Boards se lekar Pipelines tak sab ek jagah.

• 03 TOOL MAPPING – GENERIC → AZURE → AWS

KAAM	GENERIC TOOL	AZURE	AWS
Project Mgmt	JIRA, Rally	Azure Boards	CodeCatalyst
Source Code	Git, GitLab, BitBucket	Azure Repos	CodeCommit
CI/CD	Jenkins, TeamCity	Azure Pipelines	CodePipeline
Artifactory	Nexus, JFrog	Azure Artifacts	CodeArtifact
Testing	JUnit, VSTest	Azure Test Plans	Device Farm
Docs / Wiki	Confluence	Azure Wiki	—

• 04 TOOL TYPE – KAUNSA KIS CATEGORY KA HAI

TYPE	MATLAB	EXAMPLE
SaaS	Ready software, bas login karke use karo	JIRA, Azure DevOps, Grafana
PaaS	Platform jisme apna code/pipeline chalta hai	Azure Pipelines, Cloud Build
IaaS	Sirf raw infra — OS se sab khud manage	Azure VM, EC2
SCM	Source/version control	Git, GitHub, Azure Repos
IaC	Code likh ke infra banate ho	Terraform, ARM, CloudFormation
FaaS	Sirf function likho, server nahi sambhalna	Azure Functions, Lambda

💡 Interview tip: koi bhi SCM (GitHub/GitLab/BitBucket) kisi bhi pipeline (Jenkins/Azure Pipelines) se connect ho sakta hai — PAT token / credentials se authenticate karke.

05 ACCESS – DO ALAG URLS, DO ALAG DUNIYA

portal.azure.com

Azure ke **cloud resources** manage karne ke liye — VM, Storage, VNet, Public IP. Sau se zyada services milti hain.

dev.azure.com

Azure **DevOps** (software delivery) ke liye — Boards, Repos, Pipelines, Test Plans, Artifacts. Ginechune 5-6 services.

06 HIERARCHY – DONO SYSTEM BILKUL ALAG HAIN

DevOps Hierarchy



Azure Portal Hierarchy



⚠ Management Group / Subscription / Resource Group ka concept sirf Azure Portal mein chalta hai — Azure DevOps mein bilkul kaam nahi karta. Dono ko mat mix karo, ye common confusion hai.

07 DONO KA KAAM – EK LINE MEIN

Azure Portal

Cloud infrastructure banata aur chalata hai (VM, DB, network)

Azure DevOps

Us infrastructure tak code pahunchane ka process manage karta hai

Connection

Service Connection ke through DevOps, Portal ke resources pe deploy karta hai

08 PROJECT KE ANDAR KYA-KYA MILTA HAI

SERVICE	KAAM
Azure Boards	Work items, sprints, bugs, backlog track karna
Azure Repos	Git repository — code rakhna, branch, PR
Azure Pipelines	CI/CD — build, test, deploy automate karna
Azure Test Plans	Manual aur automated test cases manage karna
Azure Artifacts	Packages/build output store karna (feed banana)
Azure Wiki	Team documentation likhna

• 09 ORGANIZATION SETUP – STEP BY STEP

Jaise VM banane ke liye pehle **Subscription → Resource Group** chahiye, waise Repo/Pipeline banane ke liye pehle **Organization → Project** chahiye. Neeche poora practical flow — "Vistara Technologies" example ke saath.

1 Sign In karo

dev.azure.com pe jao, Microsoft/Gmail account se sign in karo. Free trial se bhi kaam ho jayega.

2 View My Organizations

Get Started page pe [View My Organizations](#) button dabao — organization banane wale page pe pahunch jaoge.

3 Create New Organization

[+ New Organization](#) pe click karo. Naam **globally unique** hona chahiye — koi aur pehle se use kar raha ho to nahi milega. e.g. "Vistara Technologies".

4 Region + Subscription link

Data kaha store hoga (e.g. India) select karo. Organization banane ke liye ek **Azure Subscription** link karna zaroori hai.

5 Project banao

[Create a Project](#) → naam do (e.g. "Vistara Sense") → **Visibility** choose karo: Private (default) ya Public.

6 Teams & Users add karo

Project Settings → Teams se team banao, phir Users add karke **Permissions** do (Reader / Contributor / Admin).

7 Repository banao

[Repos → New Repository](#). Do raste: **New** (khali repo) ya **Import Repository** (GitHub ka URL de do, seedha clone ho jayega — local clone-push ka jhanjhat nahi).

8 Main branch pe Protection Policy

Direct push band karo — sirf Pull Request se merge ho, kam se kam 1 reviewer ke approval ke baad. Ye production ka standard hai.

💡 Interview recall: **Organization → Project → (Boards / Repos / Pipelines / Test Plans / Artifacts)** — yehi hierarchy baar-baar poocha jata hai.

• 10 AGENT – PIPELINE KO "COMPUTER" CHAHIYE

Jaise tumko kaam karne ke liye computer chahiye, waise **pipeline ko kaam karne ke liye Agent** chahiye. Pipeline sirf ek "robot" hai jo Git se connected hai aur steps follow karta hai — actual commands **Agent** pe chalti hain.



Agent pe wo tool hona zaroori hai jo chalana hai — Git chahiye to Git, Terraform chahiye to Terraform install hona chahiye.

Microsoft Hosted Agent

Microsoft ka temporary cloud machine. Kuch setup nahi — turant use. **"Rent pe ghar"** jaisa. Har run pe fresh machine milti hai.

Self Hosted Agent

Apna computer/VM registered as agent. Zyada control, custom tools, cache bachta hai. **"Apna ghar"** jaisa — production mein zyada use.

• 11 SELF-HOSTED AGENT ATTACH KARNE KE STEPS

#	STEP
1	Project Settings → Agent Pools → Default → New Agent
2	OS choose karo (Windows / Linux / macOS), download link milega
3	User Settings → Personal Access Token (PAT) banao
4	Agent folder extract karo, <code>config.cmd</code> (ya <code>./config.sh</code>) chalao
5	Organization URL + PAT token daalo, agent pool select karo
6	Service ke roop mein run karo → Agent Pools mein status Online dikhna chahiye

• 12 PIPELINE STRUCTURE – 5 LEVELS

LEVEL 1

Stages

LEVEL 2

Jobs

LEVEL 3

Steps

LEVEL 4

Tasks

LEVEL 5

Commands

Factory analogy: Pipeline = poori factory · Stage = ek department · Job = team ka kaam · Step = worker ka action · Command = actual kaam.

13 PRACTICAL TERRAFORM PIPELINE FLOW

Stage 1 – Init & Plan

```
git clone → cd preprod/ → install terraform → terraform init → fmt → validate →
install az cli → az login → terraform plan
```

Stage 2 – Apply (main branch only)

```
Manual Approval → terraform apply -auto-approve
```

Feature branch se sirf **Plan** tak chalega. Main branch se **Plan + Apply** dono. Isse koi galti se production mein apply nahi kar sakta.

14 STATE FILE – STORAGE ACCOUNT MEIN KYUN RAKHTE HAIN

VM/DB banate waqt credentials (ID/password) **state file** mein save ho jaate hain — isliye Git pe kabhi push nahi karte. State file **Storage Account (blob container)** mein rakhi jaati hai, jise **remote backend** kehte hain.

REASON	KYA FAAYDA
Security	RBAC se sirf limited logo ko access
Locking	Do log ek saath apply nahi kar payenge
Versioning	Purane version pe rollback ho sakta hai
Replication	LRS / ZRS / GRS — region-level backup
SLA	99.99% availability Microsoft deta hai

15 GIT WORKFLOW – POORA CYCLE

```
feature branch → push → pipeline trigger → scan stage → plan stage → PR raise →
review + merge → approval → apply
```

Scan stage mein security/quality tools chalte hain: **tfsec, checkov, terratest, tfint**.

Merge conflict: do log ek hi line pe alag change karein to aata hai — discuss karke manually resolve karna padta hai.

• 16 YAML KYA HAI – AUR KYUN HAR JAGAH HAI

YAML **code nahi, configuration language** hai. Iska kaam hai machine ko batana "kya karna hai" — insaan ke padhne layak format mein. Isiliye **Azure Pipelines, Kubernetes, Docker Compose, Ansible, GitHub Actions** — sab YAML use karte hain.

Sabse zaroori baat: YAML ko **ratna nahi hai**. Sirf 4 rules aur ek hierarchy samajh lo — baaki sab wahi cheez repeat hoti hai. Syntax tool ke hisaab se badalta hai, concept same rehta hai.

• 17 RULE 1 – KEY: VALUE

```
country: India
job: DevOps
```

Colon `:` key aur value ko alag karta hai. Colon ke **baad ek space zaroori** hai — `key:value` galat, `key: value` sahi.

• 18 RULE 2 – INDENTATION = HIERARCHY

Ye YAML ka **sabse important** rule hai. Jo cheez andar (right) hai, wo upar wali ke **andar** hai.

```
person:
  name: Neeraj
  role: DevOps
# name aur role dono person ke andar hain
```

⚠️ YAML mein **TAB kabhi mat use karo** — sirf spaces (normally 2 per level). Tab se pipeline fail ho jayega. Ye **sabse common error** hai.

• 19 RULE 3 – LIST KE LIYE HYPHEN

```
tools:
- Git
- Docker
- Terraform
```

Pipeline mein **stages, jobs, steps** — ye sab lists hain, isiliye har ek ke aage hyphen lagta hai.

• 20 RULE 4 – DATA TYPES

TYPE	EXAMPLE	NOTE
String	name: Neeraj	quotes optional (special char ho to lagao)
Number	replicas: 3	bina quotes ke
Boolean	enabled: true	true / false — lowercase
List	- Git	hyphen se
Map	person:	andar indented keys

• 21 PIPELINE YAML KA SKELETON – YAAD KAR LO

```
trigger:
  - main
pool:
  vmImage: ubuntu-latest
stages:
  - stage: Validate
    jobs:
      - job: ValidateJob
        steps:
          - script: terraform init
          - script: terraform fmt -check
          - script: terraform validate
```

Har naya level **2 space andar** jata hai. "Unexpected value" error aaye to 90% chance indentation ki galti hai.

• 22 KEY KA MATLAB – EK LINE MEIN

trigger:	Kis branch pe push hone par pipeline chale
pool:	Kaunsa agent use hoga (MS-hosted ya self-hosted)
stages:	Stage ki list — yahan se hierarchy shuru
jobs:	Us stage ke andar kaam ke group
steps:	Job ke andar actual commands

• 23 TUMHARA PRACTICE PROJECT – 3-STAGE TERRAFORM PIPELINE

Folder structure: `main.tf` · `variables.tf` · `outputs.tf` · `azure-pipelines.yml`

STAGE	JOB	STEPS
1. Validate	ValidateJob	terraform init → fmt -check → validate
2. Plan	PlanJob	az login → terraform plan
3. Apply	ApplyJob	■ manual approval → terraform apply

• 24 PRACTICE STEPS – ISI ORDER MEIN KARO

- 1 Pehle sirf 1 stage banao**
Sirf Validate stage, teen script steps. Chalao aur green dekho — ek baar chal gaya to aadha darr khatam.
- 2 Jaan-bujh ke indentation todo**
Ek line 2 space aage-peeche karke chalao, error dekho. **Ye sabse achhi practice hai** — real job mein rozana kaam aayegi.
- 3 Doosra stage add karo (Plan)**
Ab dikhega ki stages ek ke baad ek chalte hain. `dependsOn` ka concept yahin clear hoga.
- 4 az login ko Service Connection se karo**
Hardcoded credentials kabhi mat rakhna. Service Connection banao — real-world best practice hai.
- 5 Apply stage + Manual Approval**
Environment banao, uspe approval check lagao. Tabhi Apply chalega — production ka actual pattern.

• 25 HOMEWORK CHECKLIST

- ☐ Azure DevOps **Organization** set karna
- ☐ **Project** create karna
- ☐ **Repo** banana — dono tarike: Import se, aur local clone-push se
- ☐ **Self-hosted agent** attach karna apne computer pe
- ☐ **Microsoft Hosted vs Self Hosted** — likh ke lana kab kaunsa use hota hai
- ☐ **3-stage YAML pipeline** banana (Validate → Plan → Apply)

• 26 SELF-TEST — BINA DEKHE YE 5 BATA PAO TO CONCEPT CLEAR HAI

- ☐ YAML kya hai aur kahan-kahan use hota hai?
- ☐ key: value mein colon ke baad space kyun zaroori hai?
- ☐ Indentation kya karta hai — aur TAB kyun mana hai?
- ☐ List ke aage hyphen kyun lagta hai?
- ☐ Pipeline → Stage → Job → Step → Task ki hierarchy kya hai?

• 27 PIPELINE MODULE — AAGE KA ROADMAP (20 POINTS)

- | | |
|--|---|
| 01 Classic vs YAML pipelines | 11 Artifacts — publish karna |
| 02 Login: hardcoded vs Service Connection | 12 Approvals & Checks |
| 03 Service Connection (App Registration) | 13 Templates (reusable pipeline) |
| 04 OIDC federated connection | 14 Parallel Jobs scenarios |
| 05 Best practice: with/without secret | 15 Pipeline production failures |
| 06 Variables in pipeline | 16 Day-to-day troubleshooting |
| 07 Variable Groups | 17 Optimization techniques |
| 08 Parameters in pipeline | 18 Agent Pools management |
| 09 Dev/Prod environment selection | 19 Branch protection policies |
| 10 Secrets management (Key Vault) | 20 GitHub Actions migration path |

• 28 QUICK INTERVIEW RECALL

Azure DevOps kya hai?	SaaS product, TFS/VSTS ka 2019 rebrand
Kaise access?	dev.azure.com (portal.azure.com se alag)
Hierarchy?	Organization → Project → Boards/Repos/Pipelines
Agent kya hota hai?	Pipeline ko commands chalane ke liye compute
State file kaha rakhte?	Storage Account — security, locking, versioning
YAML mein TAB?	Bilkul nahi — sirf spaces, warna pipeline fail